DAVIDEV

Portfólio Profissional

Documento de Referência para PRD e Arquitetura

Um portfólio desenvolvido como um produto de software completo, com blog, CMS administrativo, API REST e aplicação prática de engenharia de software assistida por IA.

Domínio planejado: davidev.net.br

Versão inicial • Setembro de 2026

1. Visão geral do produto

O DaviDev será um portfólio profissional criado para apresentar a trajetória, experiências, habilidades, conhecimentos e projetos do desenvolvedor. Além da vitrine profissional, o produto terá um blog técnico próprio, permitindo publicar conteúdo para compartilhar conhecimento com outras pessoas.

O projeto não será tratado como um simples site institucional. Ele será construído como um produto de software completo, utilizando um fluxo estruturado de engenharia de software, desenvolvimento assistido por IA, MCP, harnesses e artefatos como PRD, arquitetura, especificações e tarefas.

Objetivo central: demonstrar simultaneamente o resultado final do trabalho e a capacidade técnica e metodológica utilizada para construí-lo.

2. Objetivos

- Apresentar profissionalmente o desenvolvedor e sua trajetória.
- Demonstrar habilidades, experiências, conhecimentos e projetos.
- Publicar artigos técnicos por meio de um blog próprio.
- Criar um painel administrativo para gerenciamento do conteúdo.
- Demonstrar domínio de desenvolvimento full-stack moderno.
- Aplicar arquitetura hexagonal no backend.
- Aplicar práticas de autenticação, segurança e documentação de APIs.
- Demonstrar um processo de desenvolvimento orientado por especificações e apoiado por IA.

3. Escopo funcional

Área pública

- Home com apresentação, destaques e acesso às principais áreas.
- Sobre mim, incluindo foto e apresentação profissional.
- Habilidades técnicas.
- Conhecimentos técnicos e conceituais.
- Experiências profissionais.
- Projetos e respectivos detalhes técnicos.
- Blog com listagem e leitura de artigos.
- Contato e links profissionais.
- Página explicando como o próprio projeto foi construído.

Painel administrativo

- Login exclusivo para administração.
- Visualização de todos os posts.
- Criação de novos posts.
- Edição de posts, inclusive posts já publicados.

- Exclusão de posts.
- Editor visual e suporte a Markdown.
- Preview do artigo antes da publicação.
- Gerenciamento de categorias e tags.
- Versionamento dos artigos.
- Possibilidade futura de gerenciamento das demais informações do portfólio.

4. Estrutura de páginas

Página	Objetivo
Home	Apresentação rápida, destaques, projetos e artigos recentes.
Sobre mim	Trajetória, apresentação profissional e foto.
Habilidades	Tecnologias e competências práticas.
Conhecimentos	Conceitos, arquitetura, fundamentos e áreas de domínio.
Experiências	Experiências profissionais, responsabilidades e tecnologias.
Projetos	Cases, desafios, tecnologias, arquitetura e links.
Blog	Listagem, filtros, categorias, tags e artigos.
Contato	Canais profissionais e formulário de contato.
Como foi construído	Case study do processo de engenharia e desenvolvimento com IA.

5. Gestão de conteúdo e blog

O blog funcionará como um CMS próprio. O administrador terá controle completo sobre os artigos, permitindo criar, editar, publicar e excluir conteúdo.

Ciclo de um post

```
Rascunho → edição → preview → publicação → edição posterior → nova versão
```

Informações mínimas do artigo

- Título.
- Slug.
- Resumo.
- Conteúdo.
- Imagem de capa.
- Categoria.
- Tags.
- Status.
- Data de publicação.
- Data da última atualização.
- Versão atual.

6. Editor e preview

A tela de cadastro/edição deverá permitir escrever o artigo e visualizar como ele será apresentado no site. O sistema deverá aceitar Markdown e também oferecer uma experiência de edição adequada para conteúdo técnico.

O preview poderá ser apresentado lado a lado com o editor ou em modo alternável (Editor / Preview / Split View). A renderização deverá refletir o mais fielmente possível o componente utilizado na página pública do artigo.

7. Versionamento dos artigos

Cada alteração relevante após a publicação deverá permitir identificar que o artigo foi atualizado. A interface pública deverá informar a data de publicação e a data da última edição quando forem diferentes.

A arquitetura deverá considerar versionamento real do conteúdo, permitindo futuramente comparar versões, consultar histórico e restaurar uma versão anterior.

8. Stack tecnológica

Camada	Tecnologia / solução
Frontend	Next.js + TypeScript
Estilização	Tailwind CSS
Componentes UI	shadcn/ui
Backend	Node.js + Fastify
API	REST
Documentação API	OpenAPI / Swagger
Validação	Zod
Client API	Orval
ORM	Drizzle ORM
Banco de dados	PostgreSQL
Provedor do banco	Neon
Autenticação	JWT
Tokens	Access Token + Refresh Token
Proteções	CORS + Rate Limiting
Hospedagem	Vercel
Domínio	davidev.net.br
Arquitetura	Hexagonal

9. Bibliotecas e plugins

- shadcn/ui
- Zod
- Orval
- eslint-plugin-simple-import-sort
- prettier-plugin-tailwindcss
- eslint-config-prettier
- Fastify + Swagger

10. Arquitetura hexagonal

O backend deverá seguir Arquitetura Hexagonal (Ports and Adapters), mantendo o domínio e os casos de uso independentes de frameworks e detalhes de infraestrutura.

> Fluxo conceitual: Frontend → API/Adapters → Application/Use Cases → Domain → Ports → Adapters de infraestrutura → Drizzle/PostgreSQL/Neon.

Princípios arquiteturais

- Regras de negócio concentradas no domínio.
- Casos de uso na camada de aplicação.
- Interfaces/ports definindo dependências externas.
- Fastify, Drizzle e PostgreSQL tratados como detalhes de infraestrutura/adapters.
- Controllers e mecanismos HTTP isolados da regra de negócio.
- Possibilidade de testar casos de uso sem depender do banco ou servidor HTTP.

11. API REST

A API será responsável por expor os recursos do blog e as operações administrativas, com documentação OpenAPI/Swagger.

Recursos iniciais

Recurso	Operações esperadas
Posts	Listar, consultar, criar, editar e excluir
Categorias	Listar, criar, editar e excluir
Tags	Listar, criar, editar e excluir
Autenticação	Login, refresh e logout

Exemplos de endpoints

```
GET /api/v1/posts
GET /api/v1/posts/:slug
POST /api/v1/posts
PUT /api/v1/posts/:id
DELETE /api/v1/posts/:id

POST /api/v1/auth/login
POST /api/v1/auth/refresh
POST /api/v1/auth/logout
```

12. Autenticação e segurança

- JWT para autenticação.
- Access Token para acesso às rotas protegidas.
- Refresh Token para renovação da sessão.
- CORS configurado para controlar origens permitidas.
- Rate Limiting para reduzir abuso e ataques automatizados.
- Rotas administrativas protegidas.
- Credencial inicial do administrador fornecida por variável de ambiente.
- A senha deverá ser armazenada no banco somente em formato de hash seguro, nunca em texto puro.

A configuração detalhada de infraestrutura, deploy, CI/CD, observabilidade e demais aspectos operacionais ficará para a etapa posterior de arquitetura/implementação, conforme definido para o projeto.

13. Banco de dados — visão inicial

O banco será PostgreSQL hospedado no Neon e acessado pelo backend por meio do Drizzle ORM.

Entidades iniciais esperadas

- Admin/User administrativo.
- Post.

- PostVersion.
- Category.
- Tag.
- Relacionamento Post–Tag.
- Possíveis entidades de Project, Experience, Skill e Knowledge.

O modelo definitivo, cardinalidades, índices, constraints e estratégia de migração deverão ser definidos no documento de arquitetura.

14. Desenvolvimento assistido por IA

Um dos diferenciais do projeto será demonstrar não apenas o produto final, mas o processo utilizado para desenvolvê-lo. O portfólio deverá servir como evidência prática de um fluxo moderno de engenharia de software com IA.

Fluxo de desenvolvimento de referência

```
Ideia → PRD → Arquitetura → Especificação → Tasks → Implementação assistida por IA → Testes → Validação/Convergência → Code Review → Deploy
```

Elementos a serem demonstrados

- Uso de IA como apoio ao ciclo de desenvolvimento.
- MCP para integração de ferramentas e contexto.
- Harnesses e automações para execução controlada.
- Especificações como fonte de verdade para implementação.
- Geração e acompanhamento de tasks.
- Validação da implementação contra especificações e arquitetura.
- Documentação das decisões técnicas.

A página 'Como este projeto foi construído' deverá transformar esse processo em um case study público, mostrando decisões e aprendizados sem expor credenciais, segredos ou informações privadas.

15. Requisitos não funcionais — direcionadores

- Código fortemente tipado com TypeScript.
- Separação clara de responsabilidades.
- API documentada e validada.
- Tratamento consistente de erros.
- Segurança adequada para área administrativa.
- Boa experiência de leitura para artigos técnicos.
- Responsividade para diferentes tamanhos de tela.
- SEO e acessibilidade considerados no frontend público.
- Testabilidade das regras de negócio.
- Manutenibilidade e facilidade de evolução.

16. Roadmap conceitual

Fase	Objetivo
1. Fundação	Estrutura do projeto, padrões, arquitetura e configuração base.
2. Portfólio	Home, Sobre, Habilidades, Conhecimentos, Experiências e Projetos.
3. Blog	API, banco, listagem e leitura de artigos.

Fase	Objetivo
4. Administração	Login, CRUD, editor, Markdown e preview.
5. Versionamento	Histórico e identificação de versões dos artigos.
6. Segurança	JWT, access/refresh token, CORS e rate limiting.
7. Qualidade	Testes, validações e refinamento.
8. Publicação	Deploy na Vercel e configuração do domínio.
9. Case study	Documentação pública do processo de desenvolvimento com IA.

17. Decisões registradas

Decisão	Definição
Produto	Portfólio profissional com blog e CMS administrativo.
Frontend	Next.js + TypeScript + Tailwind + shadcn/ui.
Backend	Node.js + Fastify.
API	REST com OpenAPI/Swagger.
Persistência	PostgreSQL/Neon + Drizzle.
Validação	Zod.
Geração de client	Orval.
Arquitetura	Hexagonal / Ports and Adapters.
Autenticação	JWT + Access Token + Refresh Token.
Proteções	CORS + Rate Limiting.
Hospedagem	Vercel.
Domínio	davidev.net.br.
Admin inicial	Credencial provisionada a partir de variável de ambiente.
Conteúdo	Editor + Markdown + Preview.
Artigos	Edição após publicação + identificação de atualização.
Versionamento	Modelo preparado para histórico de versões.

18. Pontos para detalhamento no PRD e na arquitetura

- Definição completa das personas e necessidades do usuário.
- Regras detalhadas de publicação, rascunho e versionamento.
- Modelo definitivo do banco de dados.
- Contrato completo da API.
- Política de expiração e rotação de tokens.
- Estratégia de armazenamento de imagens.
- Política de soft delete e recuperação.
- Estratégia de renderização e sanitização do Markdown.
- Permissões e escopo da área administrativa.
- Estratégia completa de testes.
- Detalhamento de deploy e operação.

19. Resumo arquitetural

O DaviDev deverá ser tratado como um produto full-stack orientado a conteúdo, com frontend público em Next.js, backend REST em Fastify, persistência PostgreSQL via Drizzle/Neon e uma área administrativa protegida por JWT. A Arquitetura Hexagonal será utilizada para manter o domínio e os casos de uso desacoplados de HTTP, banco e frameworks.

O produto também funcionará como um projeto demonstrativo de engenharia de software: as decisões, especificações, implementação e evolução serão organizadas em um fluxo que incorpora IA, MCP e automações, permitindo que o próprio portfólio seja uma demonstração do conhecimento técnico que ele apresenta.

Documento de referência inicial — destinado a servir de base para a elaboração do PRD e do Documento de Arquitetura. Decisões de implementação não definidas aqui deverão ser refinadas nas etapas posteriores.